Campaign Rule Engine — Cursor Implementation Plan

Project Overview

A full-featured Campaign Creation & Rule Engine UI for business users to configure campaigns for off-roll agents/employees. Built with React + TypeScript, featuring multi-step wizard flows, complex rule builders, and payout equation editors.

Tech Stack

Layer	Choice	Reason
Framework	React 18 + TypeScript	Type safety for complex rule structures
Styling	Tailwind CSS + shadcn/ui	Rapid, consistent component library
State Management	Zustand	Lightweight, clean for wizard/multi-step state
Forms	React Hook Form + Zod	Validation for complex nested rule forms
Tables	TanStack Table	For agent listing, rule display
Charts	Recharts	Campaign performance previews
File Upload	react-dropzone	Excel upload for agent targeting
Excel Parsing	xlsx (SheetJS)	Parse uploaded agent files
Date Handling	date-fns	Campaign period management
Drag & Drop	@dnd-kit/core	Reordering rules, payout conditions
Router	React Router v6	Multi-page navigation
Icons	Lucide React	Consistent icon set

Folder Structure

```
src/
├── components/
│   ├── campaign/
│   │   ├── wizard/
│   │   │   ├── Step1_CampaignDetails.tsx
```

```
│   │   │   ├── Step2_Targeting.tsx
│   │   │   ├── Step3_QualifyingRules.tsx
│   │   │   ├── Step4_PayoutStructure.tsx
│   │   │   └── Step5_ReviewSubmit.tsx
│   │   ├── rule-builder/
│   │   │   ├── RuleSetBuilder.tsx
│   │   │   ├── RuleRow.tsx
│   │   │   ├── OperatorSelector.tsx
│   │   │   ├── ValueInput.tsx
│   │   │   └── RuleGroupContainer.tsx
│   │   ├── payout/
│   │   │   ├── PayoutBuilder.tsx
│   │   │   ├── KPIBlock.tsx
│   │   │   ├── VintageConditionBlock.tsx
│   │   │   ├── TransactionTierTable.tsx
│   │   │   ├── ConditionalPayoutEditor.tsx
│   │   │   └── PayoutEquationPreview.tsx
│   │   ├── versioning/
│   │   │   ├── RuleVersionTimeline.tsx
│   │   │   ├── VersionCard.tsx
│   │   │   └── AddVersionModal.tsx
│   │   └── targeting/
│   │       ├── HierarchySelector.tsx
│   │       ├── ChannelSubChannelDesignation.tsx
│   │       └── ExcelUploader.tsx
│   ├── shared/
│   │   ├── WizardProgress.tsx
│   │   ├── ConditionBuilder.tsx
│   │   ├── DateRangePicker.tsx
│   │   ├── TagInput.tsx
│   │   └── ConfirmDialog.tsx
│   └── layout/
│       ├── AppShell.tsx
│       ├── Sidebar.tsx
│       └── TopBar.tsx
├── pages/
│   ├── Dashboard.tsx
│   ├── CampaignList.tsx
│   ├── CampaignCreate.tsx
│   ├── CampaignDetail.tsx
│   └── CampaignEdit.tsx
├── store/
│   ├── campaignWizardStore.ts
│   ├── campaignListStore.ts
│   └── ruleVersionStore.ts
├── types/
│   ├── campaign.types.ts
```

```
│   ├── rule.types.ts
│   ├── payout.types.ts
│   └── targeting.types.ts
├── hooks/
│   ├── useCampaignWizard.ts
│   ├── useRuleBuilder.ts
│   └── usePayoutBuilder.ts
├── utils/
│   ├── ruleEvaluator.ts
│   ├── payoutCalculator.ts
│   ├── excelParser.ts
│   └── validators.ts
└── constants/
    ├── fieldParameters.ts
    ├── operators.ts
    └── campaignTypes.ts
```

Phase 1 — Type Definitions & Data Models

Step 1.1 — Core Types (`src/types/`)

`campaign.types.ts`

typescript

```typescript
export type CampaignType = 'points' | 'cash' | 'voucher' | 'mixed';
export type CampaignStatus = 'draft' | 'active' | 'paused' | 'expired';

export interface Campaign {
  id: string;
  name: string;
  description: string;
  type: CampaignType;
  status: CampaignStatus;
  startDate: string;           // ISO date
  endDate: string;
  targeting: Targeting;
  ruleVersions: RuleVersion[]; // multiple versions within period
  createdAt: string;
  updatedAt: string;
}

export interface RuleVersion {
  id: string;
  label: string;               // e.g. "Phase 1: Oct 1–5"
  startDate: string;
  endDate: string;
  qualifyingRules: RuleGroup;
  payoutStructure: PayoutStructure;
  createdAt: string;
}
```

rule.types.ts

typescript

```typescript
export type FieldParameter =
  | 'zone' | 'city' | 'product_flag' | 'product_group'
  | 'bucket' | 'amount_collected' | 'vintage' | 'loan_amount'
  | 'transaction_count' | 'disbursal_count' | 'customer_type'
  | 'emandate_count' | 'qr_scan_count' | 'optimus_download_count';

export type Operator = '=' | '!=' | '>' | '>=' | '<' | '<=' | 'between' | 'in' | 'no

export interface RuleCondition {
  id: string;
  field: FieldParameter;
  operator: Operator;
  value: string | number | [number, number] | string[]; // single, range, or list
}

export interface RuleGroup {
  id: string;
  logic: 'AND' | 'OR';
  conditions: (RuleCondition | RuleGroup)[];  // nested groups supported
}
```

payout.types.ts

typescript

```ts
export interface PayoutStructure {
  kpis: KPI[];
  baseKPI?: string;            // ID of the gating KPI (e.g. disbursal)
  baseKPITargetByVintage?: VintageTarget[];
}

export interface KPI {
  id: string;
  name: string;                // e.g. "OEPT", "Disbursal", "Emandate"
  isBase: boolean;
  unlockCondition?: UnlockCondition;  // gate to access this KPI
  tiers: PayoutTier[];
  conditions: KPICondition[];  // e.g. "must do > 30 emandates"
}

export interface PayoutTier {
  id: string;
  dimensions: TierDimension[]; // e.g. zone=East, vintage=0-3, txn=1
  points: number;
  cashAmount?: number;
}

export interface TierDimension {
  field: FieldParameter;
  operator: Operator;
  value: string | number | [number, number];
}

export interface KPICondition {
  type: 'min_count' | 'min_amount' | 'percentage_fallback';
  threshold: number;
  fallbackPercentage?: number;  // e.g. 10% of full points if target not met
  fallbackPoints?: number;
}

export interface VintageTarget {
  vintageRange: [number, number];  // months, e.g. [0, 3]
  targetCount: number;
}

export interface UnlockCondition {
  kpiId: string;               // must complete this KPI first
  threshold: number;           // e.g. > 50 disbursals
  operator: Operator;
}
```

`targeting.types.ts`

typescript

```typescript
export type TargetingMode = 'hierarchy' | 'excel_upload';

export interface Targeting {
  mode: TargetingMode;
  hierarchy?: HierarchySelection;
  uploadedAgents?: AgentRecord[];
  uploadFileName?: string;
}

export interface HierarchySelection {
  channels: string[];
  subChannels: string[];
  designations: string[];
}

export interface AgentRecord {
  agentId: string;
  name: string;
  channel: string;
  subChannel: string;
  designation: string;
  zone: string;
  vintage: number;
}
```

Phase 2 — Constants & Config

Step 2.1 — `src/constants/fieldParameters.ts`

typescript

```
export const FIELD_PARAMETERS = [
  { value: 'zone', label: 'Zone', type: 'enum', options: ['East','West','North','Sou
  { value: 'city', label: 'City', type: 'text' },
  { value: 'vintage', label: 'Vintage (months)', type: 'number' },
  { value: 'product_flag', label: 'Product Flag', type: 'enum', options: ['NTB','ETB
  { value: 'product_group', label: 'Product Group', type: 'enum', options: ['Consume
  { value: 'bucket', label: 'Bucket', type: 'enum', options: ['X','1','2','3'] },
  { value: 'amount_collected', label: 'Amount Collected', type: 'number' },
  { value: 'loan_amount', label: 'Loan Amount', type: 'number' },
  { value: 'transaction_count', label: 'Transaction Count', type: 'number' },
  { value: 'disbursal_count', label: 'Disbursal Count', type: 'number' },
  { value: 'customer_type', label: 'Customer Type', type: 'enum', options: ['NTB','E
  { value: 'emandate_count', label: 'eMandate Count', type: 'number' },
  { value: 'qr_scan_count', label: 'QR Scan Count', type: 'number' },
];

export const OPERATORS_BY_TYPE = {
  number: ['=', '!=', '>', '>=', '<', '<=', 'between'],
  text: ['=', '!=', 'in', 'not_in'],
  enum: ['=', '!=', 'in', 'not_in'],
};
```

Step 2.2 — `src/constants/campaignTypes.ts`

typescript

```
export const CAMPAIGN_TYPES = [
  { value: 'points', label: 'Points-Based', icon: 'Star', description: 'Award points
  { value: 'cash', label: 'Cash Incentive', icon: 'DollarSign', description: 'Direct
  { value: 'voucher', label: 'Voucher Reward', icon: 'Gift', description: 'Issue gif
  { value: 'mixed', label: 'Mixed (Points + Cash)', icon: 'Layers', description: 'Co
];

export const HIERARCHY_DATA = {
  channels: ['Direct Sales', 'DSA', 'Connector', 'Branch'],
  subChannels: {
    'Direct Sales': ['In-House', 'Freelancer'],
    'DSA': ['National DSA', 'Regional DSA'],
    'Connector': ['Individual', 'Firm'],
    'Branch': ['Urban', 'Rural', 'Semi-Urban'],
  },
  designations: ['Agent', 'Senior Agent', 'Team Leader', 'Area Manager'],
};
```

Phase 3 — State Management (Zustand)

Step 3.1 — `src/store/campaignWizardStore.ts`

```typescript
import { create } from 'zustand';

interface WizardState {
  currentStep: number;          // 1-5
  campaign: Partial<Campaign>;
  activeVersionIndex: number;

  // Actions
  setStep: (step: number) => void;
  updateCampaign: (data: Partial<Campaign>) => void;
  addRuleVersion: (version: RuleVersion) => void;
  updateRuleVersion: (index: number, version: Partial<RuleVersion>) => void;
  removeRuleVersion: (index: number) => void;
  resetWizard: () => void;
}
```

Step 3.2 — `src/store/campaignListStore.ts`

- Store for CRUD of all campaigns
- Filter by status, type, date
- Pagination state

Phase 4 — Wizard Shell & Navigation

Step 4.1 — `src/components/shared/WizardProgress.tsx`

- 5-step horizontal stepper with labels
- Steps: **Details → Targeting → Qualifying Rules → Payout → Review**
- Clickable completed steps
- Current step highlighted, future steps dimmed
- Visual connector lines between steps
- Show step completion checkmarks

Step 4.2 — `src/pages/CampaignCreate.tsx`

typescript

```
// Main wizard orchestrator
// Renders the correct step component based on currentStep
// Handles Next/Back navigation
// Guards: cannot proceed without completing required fields
// Auto-saves draft to localStorage on each step change
```

Phase 5 — Step 1: Campaign Details

`src/components/campaign/wizard/Step1_CampaignDetails.tsx`

Fields to build:

1. **Campaign Name** — text input, required, max 100 chars
2. **Description** — rich textarea, max 500 chars, character counter
3. **Campaign Type** — visual card selector (4 options from constants)
4. **Campaign Period** — dual date picker (start + end date), date range validation
5. **Campaign ID** — auto-generated, read-only display

UI Details:

- Campaign type cards with icons, hover effects, selected state with border highlight
- Date range picker with min/max constraints (start < end, start >= today for new campaigns)
- Inline validation with error messages
- Preview chip showing "Oct 1 – Oct 15, 2024 (15 days)"

Phase 6 — Step 2: Agent Targeting

`src/components/campaign/targeting/`

`HierarchySelector.tsx`

- Three cascading multi-select dropdowns: **Channel → Sub Channel → Designation**
- Sub Channel options filtered by selected Channels
- "Select All" toggle per level
- Shows count badge: "3 channels, 5 sub-channels, 2 designations selected"
- Renders a summary chip list of all selections below

`ExcelUploader.tsx`

- Drag-and-drop zone (react-dropzone)

- Accepts `.xlsx` / `.csv` only
- On upload: parse with SheetJS → validate columns (agentId, name, channel, etc.)
- Show preview table of first 10 rows
- Error state for missing columns or malformed data
- Show total agent count: "847 agents loaded"
- Download template button

`Step2_Targeting.tsx`

- Toggle between "Use Hierarchy" and "Upload Excel" (Tab or Radio button)
- Renders the appropriate sub-component
- Validation: at least one selection/upload required to proceed

---

Phase 7 — Step 3: Qualifying Rules

`src/components/campaign/rule-builder/`

`RuleSetBuilder.tsx` **— Main Container**

- Top-level AND/OR logic toggle
- "Add Condition" button → appends a new `RuleRow`
- "Add Group" button → appends a nested `RuleGroupContainer`
- Empty state: "No rules defined — all transactions will qualify"
- Preview panel: human-readable rule summary at bottom

`RuleRow.tsx` **— Single Condition**

Layout: `[Field Dropdown] [Operator Dropdown] [Value Input(s)] [Delete Button]`

- **Field Dropdown**: searchable, grouped by category (Geography, Product, Transaction, Agent)
- **Operator Dropdown**: dynamically filtered based on field type (number vs enum vs text)
- **Value Input**: smart component that renders:
  - Single text/number input for `=`, `!=`, `>`, `>=`, `<`, `<=`
  - Two inputs (min–max) for `between`
  - Tag/chip multi-input for `in`, `not_in`
  - Enum dropdown (checkboxes) when field has predefined options
- Drag handle on left for reordering (dnd-kit)

`RuleGroupContainer.tsx` — **Nested Group**

- Indented card with AND/OR toggle header
- Contains its own list of `RuleRow` + nested `RuleGroupContainer`
- Collapsible with expand/collapse toggle
- Color-coded depth levels (visual nesting indicator)

Human-Readable Preview

```
WHERE zone IN [East, West, North]
  AND vintage BETWEEN 0 AND 3
  AND product_group = "Consumer Durable"
```

Phase 8 — Step 4: Payout Structure (Most Complex)

Architecture

The payout builder has two modes selectable at the top:

1. **Transaction-Based** (Campaign Example 1 — real-time, per-transaction)
2. **Target-Based** (Campaign Example 2 — cumulative target with unlock gates)

Mode 1: Transaction-Based Payout

`KPIBlock.tsx`

- KPI name input (e.g. "OEPT", "Collections")
- "Add Tier" button → opens tier definition row

`TransactionTierTable.tsx`

Visual matrix-style table where rows = dimension combinations, columns = transaction numbers

Zone	Vintage	Txn 1	Txn 2	Txn 3
East	0-3 mo	300	400	500
West	0-3 mo	300	400	500
North	0-3 mo	300	400	500
East	3+ mo	200	350	450

- "Add Dimension" dropdown to add Zone/Vintage/Transaction number as table axes

- Inline editable cells for point values
- "Add Row" to add new dimension combinations
- Export/import rows via CSV

Mode 2: Target-Based Payout with Unlock Gates

`PayoutBuilder.tsx` — **Orchestrator**

Renders:

1. **Base KPI Section** (required, unlocks further KPIs)
2. **Secondary KPIs Section** (unlocked after base KPI target met)
3. Visual flow: Base KPI → [meets target?] → Secondary KPIs unlocked

Base KPI Block

- KPI Name selector
- **Vintage Target Table:**

```
| Vintage Range | Target Count |
| 0–3 months    | 30           |
| 3+ months     | 50           |
```

"Add Vintage Band" button

- **Points-per-Tier Table** (by loan amount / customer type):

```
| Vintage  | Customer Type | Loan Amount | Points |
| 0–3 mo   | NTB           | 50,000      | 500    |
| 0–3 mo   | NTB           | 30,000      | 300    |
| 0–3 mo   | NTB           | 10,000      | 100    |
| 3+ mo    | NTB           | 50,000      | 700    |
```

`UnlockConditionEditor.tsx`

- Renders between Base KPI and Secondary KPIs
- Visual: "Agent must achieve: [Base KPI] [operator] [threshold] to unlock secondary KPIs"
- Example: "Disbursal Count > 50"

Secondary KPI Blocks (repeatable)

Each secondary KPI gets its own `KPIBlock` with:

- KPI name + description
- **Conditions Section** (`KPIConditionEditor.tsx`):
  - "Minimum threshold to earn full points": e.g. `emandate_count >= 30`
  - "Fallback rule if threshold not met": e.g. "Award 10% of defined points per unit"
  - Toggle: "Enable fallback" checkbox
- **Points-per-Unit**: simple input — "50 points per eMandate"
- Add multiple secondary KPIs via "Add KPI +" button

`PayoutEquationPreview.tsx`

- Live human-readable preview of the full payout logic:

```
BASE KPI: Disbursal
  • Vintage 0–3 months → Target: 30 disbursals
    - NTB, ₹50,000 → 500 pts | ₹30,000 → 300 pts | ₹10,000 → 100 pts
  • Vintage 3+ months → Target: 50 disbursals
    - NTB, ₹50,000 → 700 pts | ₹30,000 → 500 pts | ₹10,000 → 300 pts

UNLOCK GATE: Disbursal Count > 50

SECONDARY KPIs (unlocked after gate):
  • eMandate: 50 pts/unit
    - Condition: >= 30 emandates → full points
    - Fallback: < 30 emandates → 10% of points per unit
  • QR Scan: [define...]
```

Phase 9 — Rule Version Management

`src/components/campaign/versioning/`

`RuleVersionTimeline.tsx`

Visual timeline spanning the campaign period:

```
[Oct 1]─────[Oct 5]─────[Oct 8]──────────[Oct 15]
  Phase 1          Phase 2        Phase 3
  (Rules A)        (Rules B)      (Rules C)
```

- Each version block is clickable → expands version detail
- "Add Version" button → opens `AddVersionModal`
- Overlapping date validation (versions must not overlap)

- Visual gaps flagged with warning: "No rules defined for Oct 6–7"

`AddVersionModal.tsx`

- Version label input
- Date range picker (constrained within campaign period, non-overlapping)
- Copy rules from existing version toggle (quick-start)
- On confirm → opens Steps 3 & 4 in modal context for that version

`VersionCard.tsx`

- Shows: version name, date range, rule count, KPI count
- Status: Draft / Active / Expired
- Actions: Edit, Duplicate, Delete, View

Phase 10 — Step 5: Review & Submit

`src/components/campaign/wizard/Step5_ReviewSubmit.tsx`

Renders a full read-only summary:

1. **Campaign Details Card** — name, type, period, description
2. **Targeting Summary Card** — mode + selection summary or agent count
3. **Rule Versions Timeline** — compact version of the timeline
4. **Per-Version Rule Preview** — expandable accordion per version showing:
   - Qualifying rules (human-readable)
   - Payout structure summary
5. **Validation Checklist**:
   - ✅ Campaign period defined
   - ✅ Targeting configured
   - ✅ All campaign days covered by rule versions
   - ⚠️ No payout defined for Version 2 (warning)
   - ❌ Version date overlap detected (error, blocks submit)
6. **Action Buttons**: Save as Draft | Activate Campaign

Phase 11 — Campaign Dashboard & List

`src/pages/CampaignList.tsx`

- Search + filter bar (by status, type, date range)
- Sortable table: Name, Type, Period, Status, Created By, Actions
- Status badges with colors
- Quick actions: View, Edit, Duplicate, Pause/Activate, Delete
- Empty state illustration

`src/pages/Dashboard.tsx`

- Summary cards: Active Campaigns, Total Agents Targeted, Campaigns Expiring Soon
- Recent campaigns list (last 5)
- Quick "Create Campaign" CTA button

`src/pages/CampaignDetail.tsx`

- Full campaign view (read-only)
- Rule version timeline
- Expandable version detail
- Edit and Duplicate buttons

Phase 12 — Layout & Navigation

`src/components/layout/AppShell.tsx`

Main layout:

- Left sidebar (collapsible)
- Top bar with breadcrumbs
- Main content area

`src/components/layout/Sidebar.tsx`

Nav items:

- Dashboard
- Campaigns (→ list)
- Create Campaign
- Settings (field param config, hierarchy config)

Phase 13 — Design System & Aesthetics

Color Palette (CSS Variables)

```css
:root {
  --bg-primary: #0F1117;
  --bg-secondary: #1A1D27;
  --bg-card: #1E2130;
  --accent-primary: #6366F1;     /* Indigo */
  --accent-secondary: #10B981;   /* Emerald (success/active) */
  --accent-warning: #F59E0B;
  --accent-danger: #EF4444;
  --text-primary: #F1F5F9;
  --text-secondary: #94A3B8;
  --border: #2D3348;
  --border-focus: #6366F1;
}
```

Typography

- **Display/Headings:** `DM Sans` (Google Fonts)
- **Body/Labels:** `Inter` (clean, readable for dense rule tables)
- **Mono (rule previews):** `JetBrains Mono`

Key UI Patterns

- **Cards:** `bg-card`, `border`, `rounded-xl`, subtle shadow
- **Wizard Steps:** horizontal stepper with connecting lines, completed = filled circle
- **Rule Rows:** alternating subtle background on hover, drag handle visible on hover
- **Payout Tables:** zebra striping, inline editable cells with focus ring
- **Version Timeline:** horizontal scrollable timeline with color-coded phase blocks
- **Badges:** pill-shaped, color by status

Phase 14 — Utilities

`src/utils/ruleEvaluator.ts`

```typescript
```

```
// Takes a RuleGroup + transaction data object
// Returns: { qualifies: boolean, matchedConditions: string[] }
export function evaluateRules(rules: RuleGroup, transaction: Record<string, unknown>
```

`src/utils/payoutCalculator.ts`

```typescript
// Takes a PayoutStructure + transaction/agent data
// Returns: { points: number, breakdown: PayoutBreakdown[] }
export function calculatePayout(structure: PayoutStructure, context: AgentContext):
```

`src/utils/excelParser.ts`

```typescript
// Parse uploaded xlsx/csv → AgentRecord[]
// Validates required columns, returns errors for malformed rows
export function parseAgentFile(file: File): Promise<{ agents: AgentRecord[], errors:
```

`src/utils/validators.ts`

- `validateCampaignDates` — start < end, no past dates for new campaigns
- `validateVersionCoverage` — all campaign days covered by at least one version
- `validateVersionOverlap` — no two versions share dates
- `validatePayoutTiers` — no duplicate dimension combos

Phase 15 — Implementation Order for Cursor

Execute in this sequence for cleanest incremental development:

Sprint 1 — Foundation

1. `npx create-react-app campaign-engine --template typescript` (or Vite)
2. Install all dependencies (Tailwind, shadcn, Zustand, React Hook Form, Zod, etc.)
3. Create all type files in `src/types/`
4. Create all constants in `src/constants/`
5. Set up React Router with placeholder pages
6. Build `AppShell`, `Sidebar`, `TopBar`

Sprint 2 — Wizard Shell + Steps 1 & 2

7. Build `WizardProgress` stepper component
8. Build `CampaignCreate` page with step routing
9. Build `Step1_CampaignDetails` — all fields + validation
10. Build `HierarchySelector` — cascading dropdowns
11. Build `ExcelUploader` — drag-drop + SheetJS parsing
12. Build `Step2_Targeting` — toggle between modes
13. Wire Zustand store for wizard state

Sprint 3 — Rule Builder (Step 3)

14. Build `RuleRow` — field/operator/value triplet
15. Build operator-to-value-input mapping logic
16. Build `RuleGroupContainer` with nested group support
17. Build `RuleSetBuilder` as main container
18. Build human-readable preview renderer
19. Build `Step3_QualifyingRules` wrapping everything

Sprint 4 — Payout Builder (Step 4)

20. Build `TransactionTierTable` — editable matrix
21. Build Base KPI block with vintage target table
22. Build `UnlockConditionEditor`
23. Build secondary KPI block with `KPIConditionEditor`
24. Build `PayoutEquationPreview` — live summary
25. Build mode toggle (Transaction-Based vs Target-Based)
26. Build `Step4_PayoutStructure` as orchestrator

Sprint 5 — Versioning

27. Build `VersionCard`
28. Build `AddVersionModal` — with date constraints
29. Build `RuleVersionTimeline` — visual horizontal timeline
30. Integrate version management into wizard flow

Sprint 6 — Review & List

31. Build `Step5_ReviewSubmit` — summary + validation checklist

32. Build `CampaignList` — table + filters
33. Build `CampaignDetail` — read-only full view
34. Build `Dashboard` — summary cards + recent list
35. Wire all stores and navigation

Sprint 7 — Polish

36. Add animations (Framer Motion for step transitions, card entrances)
37. Add loading/skeleton states
38. Add toast notifications for save/error states
39. Add empty state illustrations
40. Accessibility audit (aria labels, keyboard nav)
41. Responsive breakpoints for tablet view

Key Edge Cases to Handle

Case	Handling
Version dates gap	Warning badge on timeline, blocks activation
Version date overlap	Error, blocks save
`between` operator with min > max	Inline validation swap or error
Excel upload with missing columns	Show column mapping UI or clear error
No secondary KPIs added	Show empty state with "Add KPI +" prompt
Fallback % > 100	Validate: max 100
Campaign dates in the past	Warning for new campaigns, allowed for edit
Duplicate KPI names	Validate uniqueness within campaign
Payout with no tiers	Warn at review step

Sample Data / Mock API

Create `src/mocks/sampleCampaigns.ts` with 2–3 complete campaigns matching the examples in the requirements (Oct collection campaign, Consumer Durable disbursal campaign) so every screen has realistic data during development.

Cursor-Specific Instructions

When prompting Cursor for each phase, use this pattern:

```
"Create [component name] in [file path].
It receives these props: [TypeScript interface].
It should [specific behavior].
Use Tailwind CSS classes only.
Reference types from src/types/[relevant file].
Reference constants from src/constants/[relevant file].
Here is the component it integrates with: [paste parent component]."
```

This ensures Cursor has full context per component without losing coherence across the large codebase.